VIT AP

# WEB TECHNOLOGIES

## CSE-4004

## MongoDB

mongo DB

By: Dr.MD.Sirajuddin

SCOPE

# MongoDB

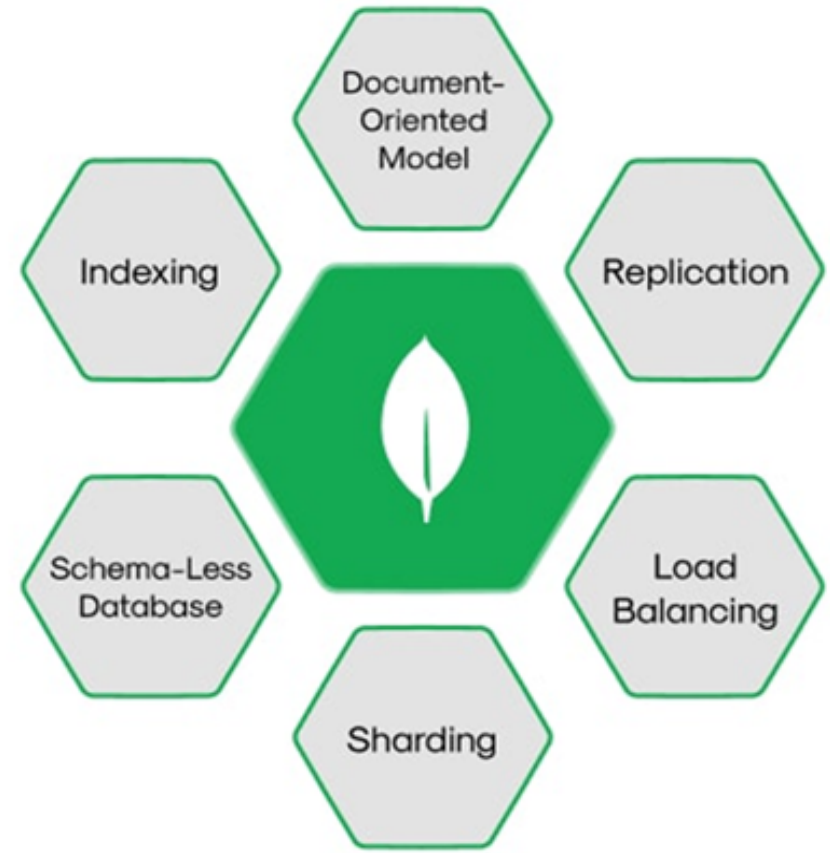

- MongoDB is NoSQL, Document oriented Database.
- Developed by MongoDB Inc. in 2009.
- Non-Relational
- Data is stored in Documents
- Provides flexible schema
- Stores data in JSON like documents (BSON- Binary JSON)

```
{
"name": “Bob",
"age": 22
}
```

- MongoDB stores data in flexible documents.
- Instead of having multiple tables you can simply keep all of your related data together. This supports fast accessing of data.

# MongoDB - Key Features

- Stores data in JSON like documents
- Replication-keeps multiple copies of your data across servers.
- Load Balancing- Client Requests are distributed across servers so no single machine gets overwhelmed.
- Sharding- Large datasets are split across multiple servers (horizontal scaling)
- Schema-Less Database- Collections don't require a fixed schema.
- Indexing- MongoDB supports various types of indexes. Indexes speed up queries dramatically, avoiding full collection scans.



# MongoDB TOOLS

- MongoDB Server (Community Edition)- DB engine runs locally.
- MongoDB Compass- GUI Tool
- MongoDB Shell (mongosh)- Command Line Interface to interact with MongoDB.
- MongoDB Atlas – Cloud Based

# MongoDB – MongoDB Atlas

- https://www.mongodb.com/products/platform/atlas-database
- Click on Get Started
- Sign-Up (email)
- Free Space of 512MB
- Set Cluster0
- Select Providers- AWS,Google Cloud, Azure
- Select Region- nearest to your location (Mumbai)
- Create Deployment
- Create User Name & Password to access database
- Select Cluster-0 → Database→ Collections

# MongoDB – CRUD Operations

## 1. Create Database

→use University_DB *(used to Select a DB)*

If it doesn't exist → MongoDB creates it when data is inserted.

## 2. Create Collection

→db.createCollection("students")

→show dbs *(you can check DB still not created)*

→db.students.insertOne({name: "Rahul", age: 20})

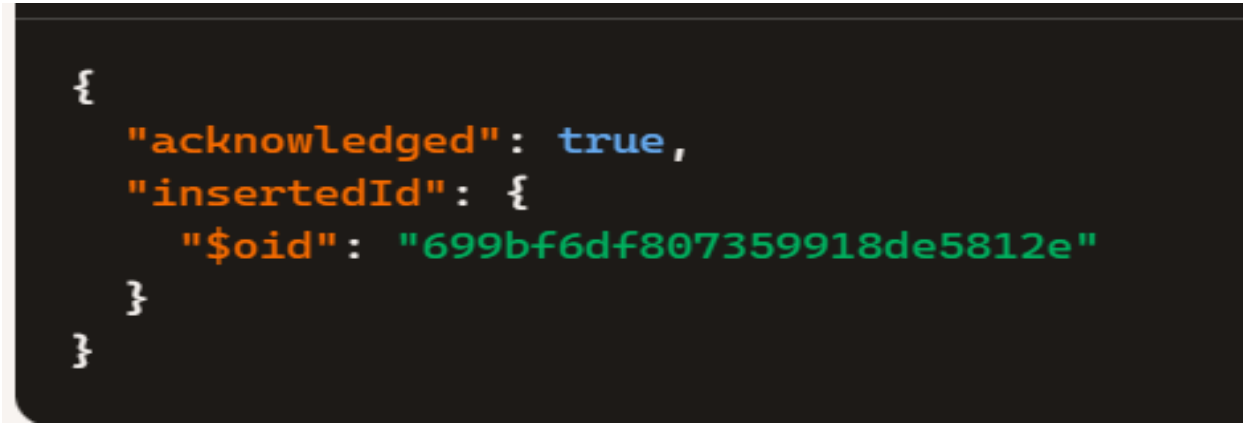
*Now the database and collection are created automatically.*

# MongoDB – CRUD Operations

- **Insert**

➢ db.students.insertOne({ name: "Aisha", age: 21, course: "BCA" });

➢ db.students.insertMany([
{name: "Ali", age: 22},
{name: "Sara", age: 23}
]);




✓ acknowledged: true → MongoDB is telling you the insert operation was accepted and completed.

✓ insertedId: This is the unique identifier (_id) MongoDB assigned to the new document. By default, MongoDB generates an ObjectId if you don't provide your own _id.

✓ oid: This is the actual ObjectId value of your new document. It's a 12 byte identifier that encodes: Timestamp, Machine identifier, Process ID etc.

# MongoDB – CRUD Operations

- Read
  - db.students.find()
  - db.students.find({age: 21})
  - db.students.find({ age: { $gte: 21 } })

- With Pretty
  - db.students.find().pretty();
  - This makes the output easier to read.

```
{
 "_id" : 1,
 "name" : "Bob",
 "age" : 22
 }

{
  "_id" : 2,
 "name" : "Alice",
 "age" : 25
 }
```

# MongoDB – CRUD Operations

- **Update**

```
db.students.updateOne( {name: "Alice"}, {$set: {age: 25}} )
db.students.updateMany( {course: "BCA"},
                        {$set: {course: "MCA"}} )
```

- **Delete**

```
db.students.deleteOne({name: "Bob"})
db.students.deleteMany({age: {$gt: 23}})
```

# MongoDB – CRUD Operations

- Delete Collection

➢db.collection_name.drop()

➢db.students.drop()

- Delete Database

1. use University_DB
2. db.dropDatabase()

Display Database

➢Show dbs

# Operators

| Arithmetic Operators | Meaning | Example |
|---|---|---|
| add | { $add: [ <Exp1>, <Exp2>] } | tSalary: { $add: ["$firstSalary", "$secondSal"] } |
| subtract | { $subtract: [ <exp1>, <exp2>] } | result: { $subtract: ["$secondSalary", "$firstSal"] } |
| multiply | { $multiply: [ <exp1>, <exp2>,] } | newSalary: { $multiply: ["$sal", 2] } |
| divide | { $divide: [ <exp1>, <exp2> ] } | halfSalary: { $divide: ["$sal", 2] } |
| abs | { $abs: <number> } | tSalary: { $abs: { $add: ["$firstSal", "$secondSal"] } } |
| floor | { $floor: <number> } | Floor_value: { $floor: "$res" } |
| ceil | { $ceil: <number> } | Ceil_value: { $floor: "$res" } |
| mod | { $mod: [ <exp1>, <exp2> ] } | {$mod: ["$side", 2]} |

# Operators

| Arithmetic Operators | Meaning | Example |
|---|---|---|
| sqrt | { $sqrt: <number> } | {squareRoot: {$sqrt: "$side"}} |
| pow | { $pow: [ <number>,<exponent> ] } | {area: {$pow: ["$side", 2]}} |
| exp | { $exp: <exponent> } | {result: {$exp: "$side"}} |
| in | { field: { $in: [<value1>, <value2>, ... <valueN>] } } | db.students.find({ name: { $in: ["John", "Bob"] } }).pretty() |

# Operators

| Comparison Operators | Meaning | Example |
|---|---|---|
| $eq | Equal | { age: { $eq: 20 } } |
| $ne | Not equal | { age: { $ne: 20 } } |
| $gt | Greater than | { age: { $gt: 18 } } |
| $gte | Greater than or equal | { age: { $gte: 18 } } |
| $lt | Less than | { age: { $lt: 30 } } |
| $lte | Less than or equal | { age: { $lte: 30 } } |
| $in | Matches any value in array | { age: { $in: [18,20,22] } } |
| $nin | Not in array | { age: { $nin: [18,20] } } |

# Operators

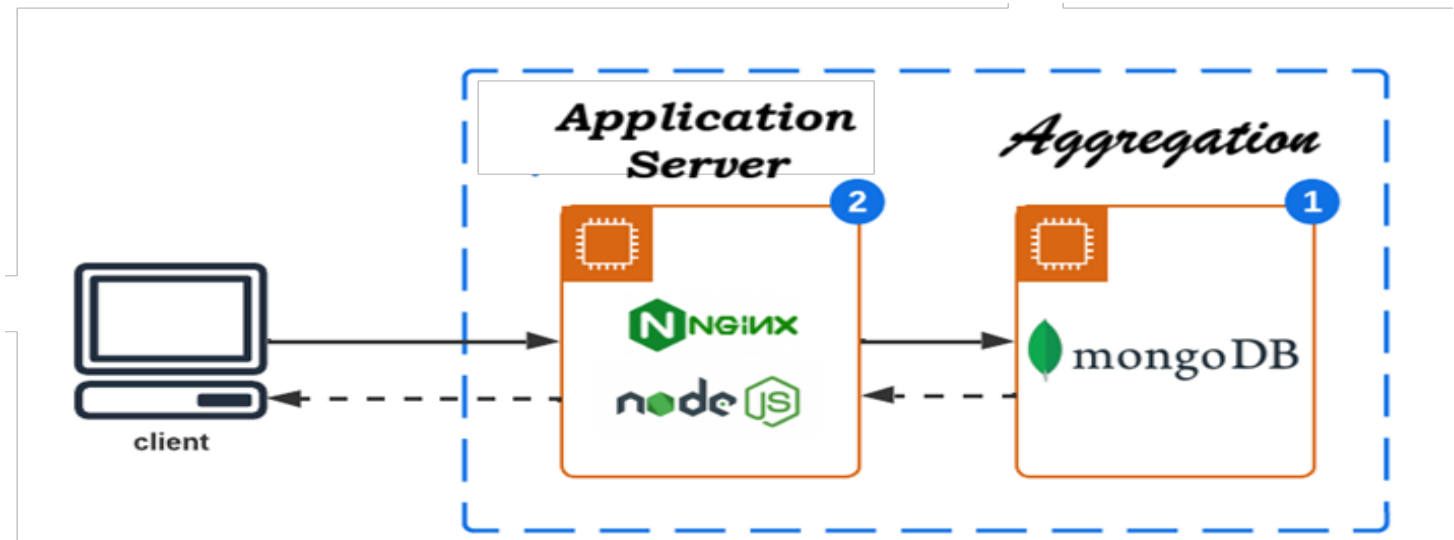
| Logical Operators | Example |
|---|---|
| $and | { $and: [{age:{$gt:18}}, {city:"Delhi"}] } |
| $or | { $or: [{age:20}, {age:25}] } |
| $not | { age: { $not: { $gt: 18 } } } |
| $nor | { $nor: [{age:20}, {city:"Delhi"}] } |

| Update Operators | Meaning |
|---|---|
| $set | Set value |
| $unset | Remove field $unset: { address: "" } |
| $inc | Increment value |
| $rename | Rename field<br>$rename: { "marks": "score" } |

| Operators | Element Operators | Meaning |
|---|---|---|
| | $exists | Field exists { age: { $exists: true } } |
| | $type | db.User.find({ age: { $type: "int"} } ) |
| Operator | | |
| $all | Matches all values, Used on Arrays<br>• hobbies: ["cricket", "reading"]<br>• db.students.find({ hobbies: { $all: ["cricket", "reading"] } }) | |
| $elemMatch | Array of objects<br>marks: [ { subject: "Math", score: 80 }, { subject: "Science", score: 90 } ]<br>db.students.find({ marks: {<br>$elemMatch: { subject: "Math", score: { $gt: 75 } }<br>} }) | |
| $size | Array size<br>db.students.find({ hobbies: { $size: 2 } }) | |

# Aggregation

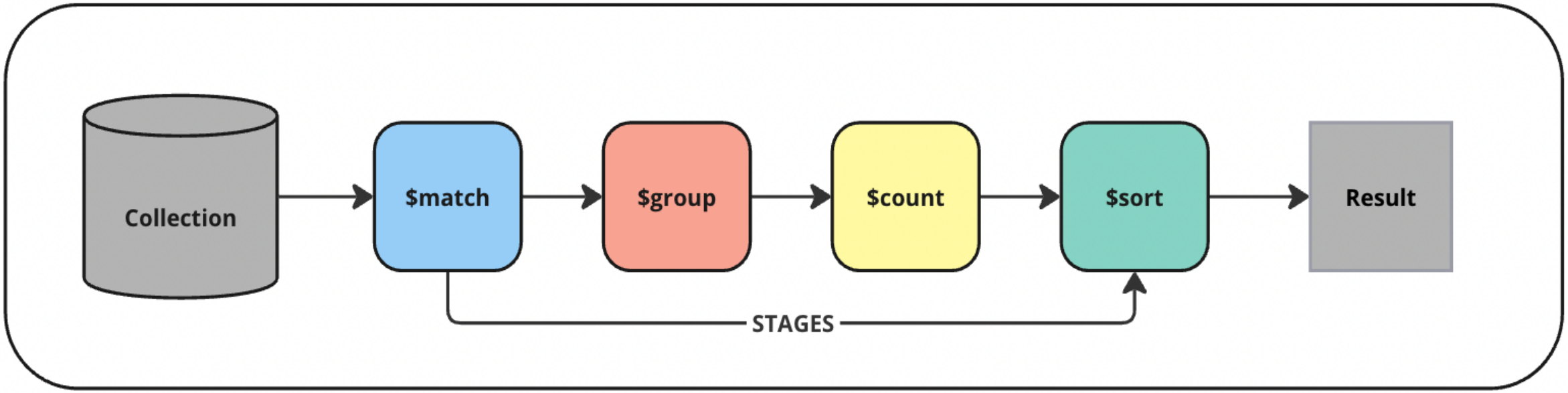
- ✓Aggregation operations process data and return computed results.
- ✓MongoDB aggregation allows data processing and computation to be performed on the database server instead of the client application, improving performance and efficiency.
- ✓MongoDB can analyze, group, calculate, and transform data inside the database and then return the final result.
  - ➢Calculate total sales
  - ➢Find average marks



| Aggregation Operators | Meaning (Process & Transform Data) | Example |
|---|---|---|
| $match | Filters documents | { $match: { department: "IT" } } |
| $group | Groups documents by a field | { $group: { _id: "$department" } } _id is grouping key |
| $sum | Calculates total | { $sum: "$salary" } |
| $avg | Calculates average | { $avg: "$salary" } |
| $min | Finds minimum value | { $min: "$salary" } |
| $max | Finds maximum value | { $max: "$salary" } |
| $count | Counts number of documents | { $count: "totalEmployees" } |
| $project | Selects specific fields | { $project: { name:1, salary:1 } } |
| $sort | Sorts documents | { $sort: { salary: -1 } } |
| $limit | Limits number of documents | { $limit: 5 } |
| $skip | Skips documents | { $skip: 5 } |

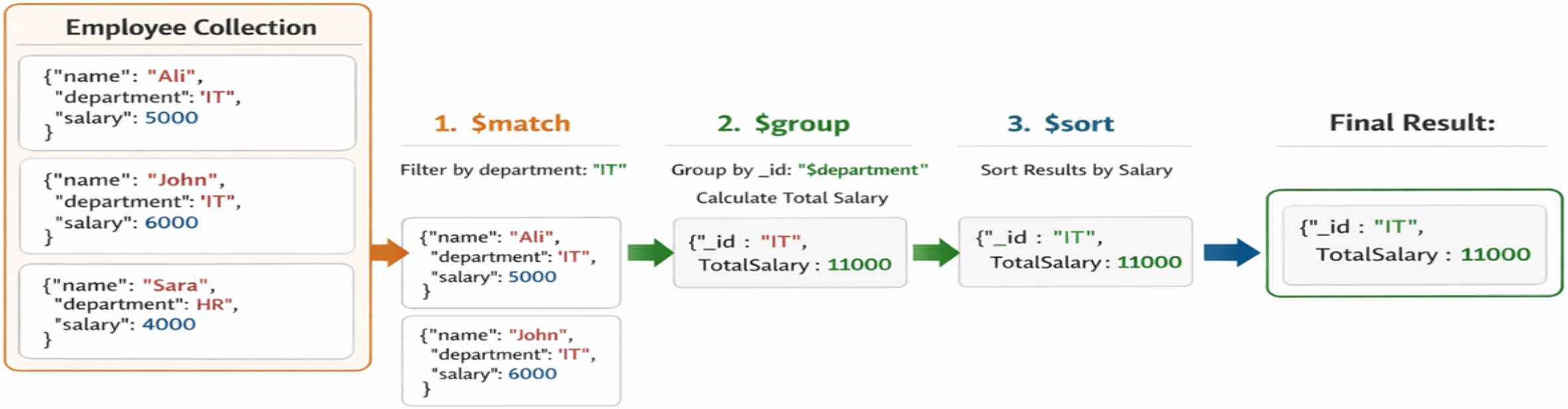
# Aggregation Pipeline

- It is a framework that Processes documents through multiple stages and transforms them step by step to produce a final result.



# Aggregation Pipeline

```
db.employees.aggregate([ { $match: {"department: "IT"}},
  { $group: {_id : "$department', totalSalary: { $sum: "$salary" }}}, { $sort: {"totalSalary: −1"}}]
```




**Output:** *Filtered Docs* → *Grouped Result* → *Sorted Result*

## Aggregate → needs an array of stages.

```
➢db.employees.aggregate([ { $match: { department: "IT" } },
{ $group: { _id: "$department", totalSalary: { $sum: "$salary" } }
},
{ $sort: { totalSalary: -1 } } ])
```

## Aggregate → needs an array of stages.

- db.employees.aggregate([ { $match: { department: "IT" } } ]);
- db.employees.aggregate([ { $group: { _id: "$department",
  totalSalary: { $sum: "$salary" }, avgSalary: { $avg: "$salary"
  } } } ]);

- db.employees.aggregate([
  { $project: { name: 1, salary: 1, _id: 0 } } ])
- *1 means include field & 0 means exclude field.*
- db.employees.aggregate([ { $count: "totalEmployees" } ])
- db.employees.aggregate([ { $match: { department: "IT" } },
  { $count: "totalEmployees" }])

**$unwind:** is an aggregation stage in MongoDB used to deconstruct an array field into multiple documents, one for each element of the array. (it flattens an array)

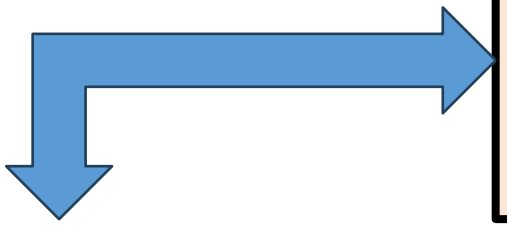
**Before $unwind**

1 document → array of 3 items

**After $unwind**

3 documents → each with 1 item

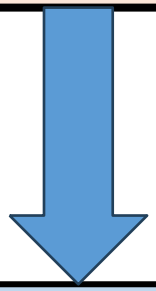
```
{ "_id": 1, "name": "Alice", "marks": [85, 90, 78] }
{ "_id": 2, "name": "Bob",   "marks": [70, 88, 92] }
{ "_id": 3, "name": "Charlie", "marks": [60, 75, 80] }
```



```
db.students.aggregate([
  { $unwind: "$marks" },
  { $group: {  _id: "$name",
      avgMarks: { $avg: "$marks" }  }   } ])
```

```
{ "_id": 1, "name": "Alice", "marks": 85 }
{ "_id": 1, "name": "Alice", "marks": 90 }
{ "_id": 1, "name": "Alice", "marks": 78 }
{ "_id": 2, "name": "Bob",   "marks": 70 }
{ "_id": 2, "name": "Bob",   "marks": 88 }
{ "_id": 2, "name": "Bob",   "marks": 92 }
{ "_id": 3, "name": "Charlie", "marks": 60 }
{ "_id": 3, "name": "Charlie", "marks": 75 }
{ "_id": 3, "name": "Charlie", "marks": 80 }
```

```
db.students.aggregate([
  { $unwind: "$marks" },
  {
    $group: {
      _id: "$name",
      avgMarks: { $avg: "$marks" }  }   } ])
```



```
{ "_id": "Alice", "avgMarks": 84.33 }
{ "_id": "Bob",   "avgMarks": 83.33 }
{ "_id": "Charlie", "avgMarks": 71.67 }
```

## Scenario

Our university wants to manage student records using MongoDB.

Database Name: University_DB

Collection Name: Students

Each student document contains:

```
{
Student_id, name, age, branch,
courses_reg: ["DBMS", "OS", "CN"],
marks: [ { subject: "DBMS", score: 85 }, { subject: "OS", score: 78 }, { subject: "CN", score: 90 } ]
}
```

## Scenario

1. Find all students from CSE branch.
2. Display only name and branch of all students.
3. Find students whose age is greater than 20.
4. Find students registered for "DBMS".
5. Find students who scored more than 80 in DBMS.
6. Count total number of students.
7. Find average marks of students.
8. Add a new course "AI" to Bob's courses.
9. Group students by branch and count them.
10. Find highest marks scored in each branch.
11. Find students who registered for more than 2 courses.

## Scenario

1. Find all students from CSE branch.

```
db.students.find({ branch: "CSE" })
```

2. Display only name and branch of all students.

```
db.students.find( { }, { name: 1, branch: 1, _id: 0 } )
```

3. Find students whose age is greater than 20.

```
db.students.find({ age: { $gt: 20 } })
```

4. Find students registered for "DBMS".

```
db.students.find({ courses_reg: "DBMS" })
```

5. Find students who scored more than 80 in DBMS.

```
db.students.find({ marks: { $elemMatch: { subject: "DBMS",
score: { $gt: 80 } } } })
```

## Scenario

6. Count total number of students.

```
db.students.countDocuments({ })
```

7. Find average marks of students.

```
db.students.aggregate([ { $unwind: "$marks" }, { $group: {
_id: null, averageMarks: { $avg: "$marks.score" } } } ])
```

8. Add a new course "AI" to Bob's courses.

```
db.students.updateOne( { name: "Bob" }, { $push: { courses_reg:
"AI" } } )
```

## Scenario

9. Group students by branch and count them.

```
db.students.aggregate([
{ $group: {
_id: "$branch",
totalStudents: { $sum: 1 } } } ])
```

10.Find highest marks scored in each branch.

```
db.students.aggregate([ { $unwind: "$marks" },
{ $group: { _id: "$branch", highestMarks: { $max: "$marks.score"
}  } } ])
```

11.Find students who registered for more than 2 courses.

```
db.students.find({ $expr: { $gt: [ { $size: "$courses_reg" }, 2 ] }
})
```

# INDEXING

- Indexing improves query processing efficiency.
- Without indexing, MongoDB must scan every document in the collection
- Indexes are special data structures that store information about the documents in a way that help MongoDB to quickly locate the right data.
- If a collection has 10,000 documents and you search for:
- db.users.find({ Roll_No: 1025 })
- Without an index on Roll_No, MongoDB will:
- Read document 1 → check Roll_No
- Read document 2 → check Roll_No
- Continue until all 10,000 documents are checked (Collection Scan)
- This makes queries slow, especially for large collections.

# CREATING INDEX

- db.students.createIndex({ Roll_No: 1 })
- MongoDB will store the index values of rollNumber in ascending order
- 1 → Ascending order
- -1 → Descending order
- MongoDB creates a separate data structure (like a sorted list) that stores: Roll_Nos
- Pointer/reference to the document location
- db.students.find({ Roll_No: 1025 })
- Looks inside the index (not the full collection)
- Quickly finds Roll_No: 1025
- Uses the stored reference to jump directly to that document

# DELETING INDEX

- db.students.dropIndex({Roll_No: 1 })
- This deletes only the index created on Roll_No.
- db.students.dropIndexes()
- removes all indexes except _id
- db.students.getIndexes()

# STORAGE CLASSES (STORAGE ARCHIECTURE)

- How data is stored, retrieved and managed on disk.
- Refers to its different pluggable storage engines, which manage how data is stored in memory and on disk
- MongoDB provides several engines to suit various performance and storage needs.

## 1. WiredTiger (WT)

- Default storage engine in modern MongoDB (Version 3.2 and above).
- Uses data compression.
- Supports document-level locking → only locks the specific document being modified.
- Provides better performance and concurrency.
- Key Features:
  i. Compression (reduces disk usage)
  ii. Efficient memory usage *(50% of RAM to serve hot data quickly)*
  iii. Crash recovery *(journal file + Checkpoints + atomic operations)*
  iv. High performance for read/write operations *(compression+caching+concurrency)*

# WiredTiger → Two Concurrent Updates

**db.serverStatus().storageEngine**

```
db.Students.updateOne(
  { _id: 1 },
  { $inc: { marks: 5 } }
)
```

Success

***WiredTiger = disk + large memory cache (50%).***

```
db.Students.updateOne(
  { _id: 1 },
  { $inc: { marks: 10 } }
)
```

- It allows both operations to proceed in parallel.
- When they both try to commit changes to the same document, one wins, the other is rolled back with a conflict error.
- WriteConflict → Rollback the failed operation

```
WriteConflict: this operation conflicted with another operation. Please retry.
```

# STORAGE CLASSES

## 2. In-Memory Storage Engine

- This engine keeps all data in RAM, entirely avoiding disk I/O to provide extremely low latency and high throughput.
- It is available in MongoDB Enterprise.
- Key Features:
  i. It is ideal for use cases that require ultra-high performance, such as real-time analytics or as a high-performance caching layer, where the entire dataset can fit in memory. (high velocity data)
  ii. Durability: While data is primarily in memory, it can be configured to take periodic snapshots to disk for crash recovery.

# STORAGE CLASSES

## 2. In-Memory Storage Engine

- The canonical copy (original copy) is held in RAM only.
- Nothing is persisted to disk (except minimal metadata).
- If the server restarts, the canonical copy disappears — meaning all data is lost.

# REPLICATION

- Replication means keeping multiple copies of the same data across different servers.
- Without replication → In case of server crash, power failure and hardware damage → All data is lost or unavailable.

## Advantages

✓High Availability: If the primary fails, one of the secondaries is automatically elected as the new primary.

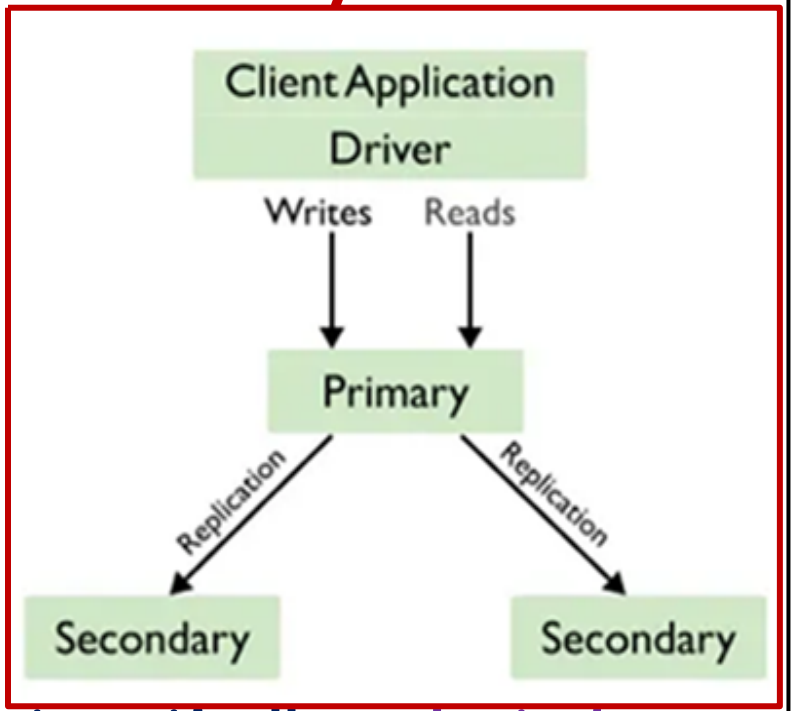
✓Fault Tolerance: Data is not lost if one server crashes.

✓Read Scaling: Can handle massive read request. Secondary nodes can serve read queries, reducing load on the primary.

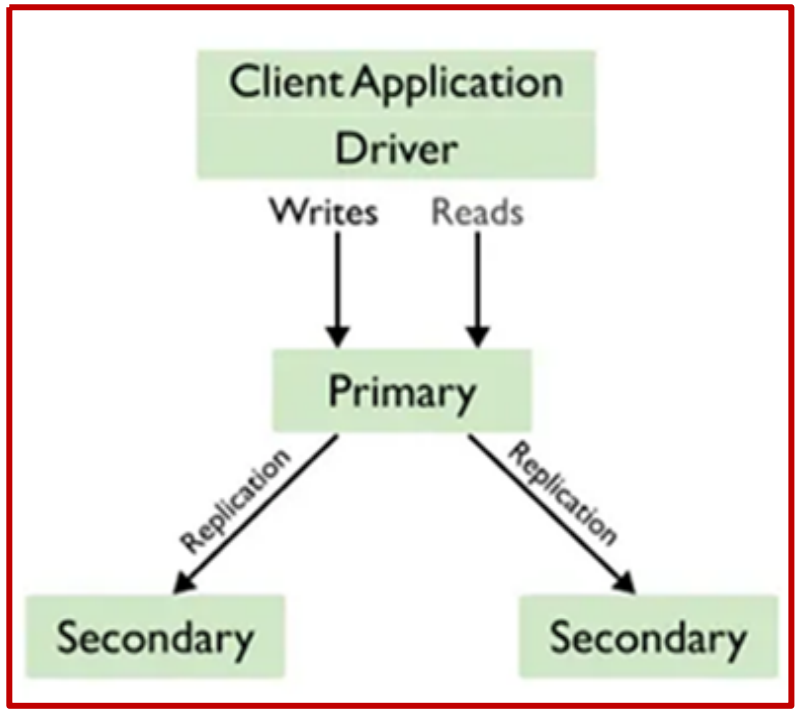
# REPLICATION

- In MongoDB, Replication is implemented using a Replica Set.
- Replica Set: A group of MongoDB servers that maintain the same dataset.
- Components of Replica Set → Primary nodes and Secondary nodes

1. Client Application Driver: connects to MongoDB
   i. It sends queries to the database.
2. Primary Node
   i. The primary is the main server in the replica set.
   ii. It handles all read operations
   iii. It handles all write operations using Oplog.rs
3. Secondary Nodes
   i. Copy data from the primary
   ii. Replicate data using Oplog to stay synchronized
   iii. Can serve read operations (if configured)
   iv. If the primary fails, one of the secondaries is automatically selected as a new primary.



# REPLICATION




```
db.Students.updateOne({ _id: 1 }, { $set: { marks: 95 } })
```

Oplog.rs

Primary

```
{
  "ts": Timestamp(123456, 1),
  "op": "u",
  "ns": "demoDB.Students",
  "o2": { "_id": 1 },
  "o": { "$set": { "marks": 95 } }
}
```

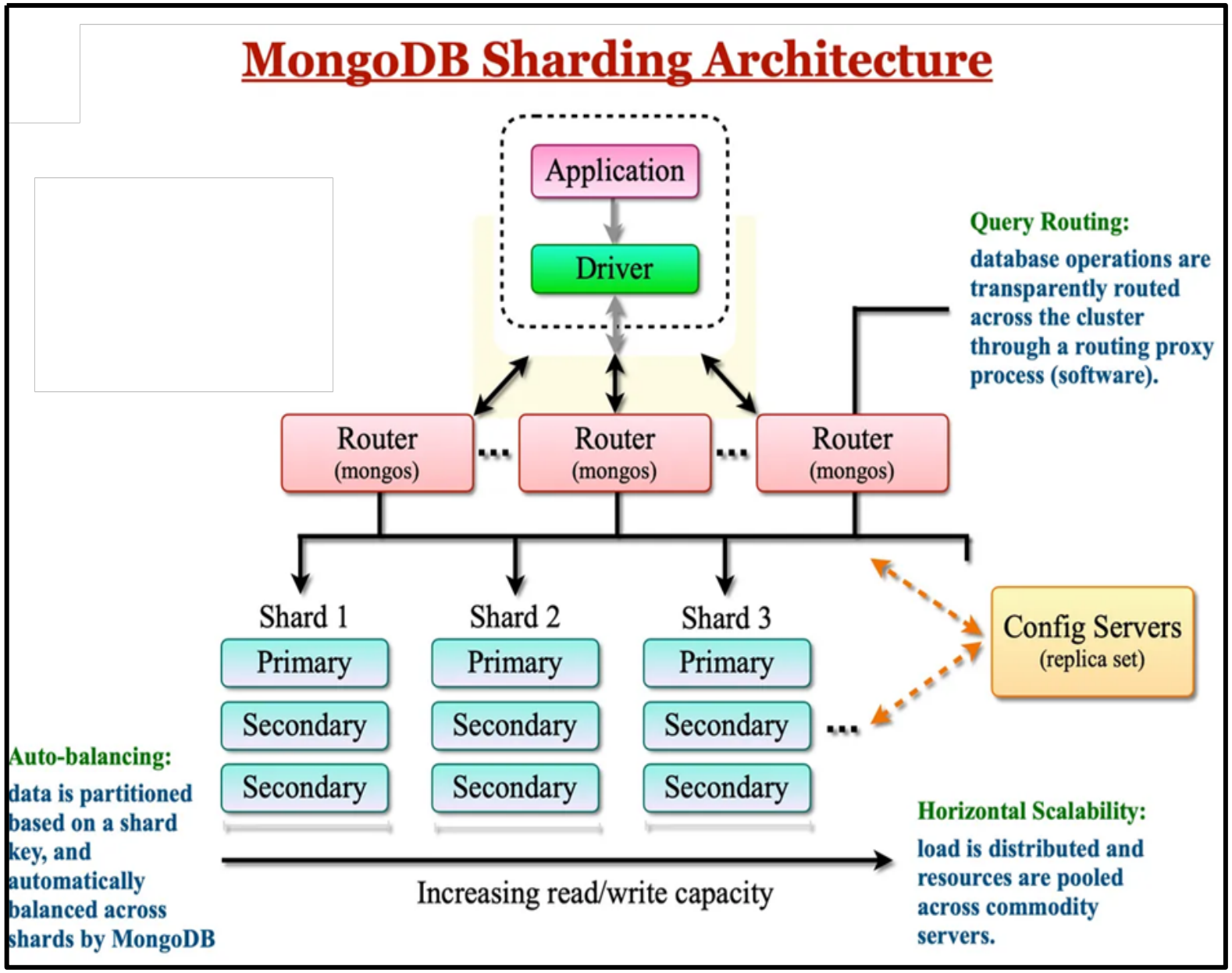
Secondary

```
{
  "ts": Timestamp(123456, 1),
  "op": "u",
  "ns": "demoDB.Students",
  "o2": { "_id": 1 },
  "o": { "$set": { "marks": 95 } }
}
```

ts: time stamp
op: operation type
ns: name space
o2: which document
o: what operation

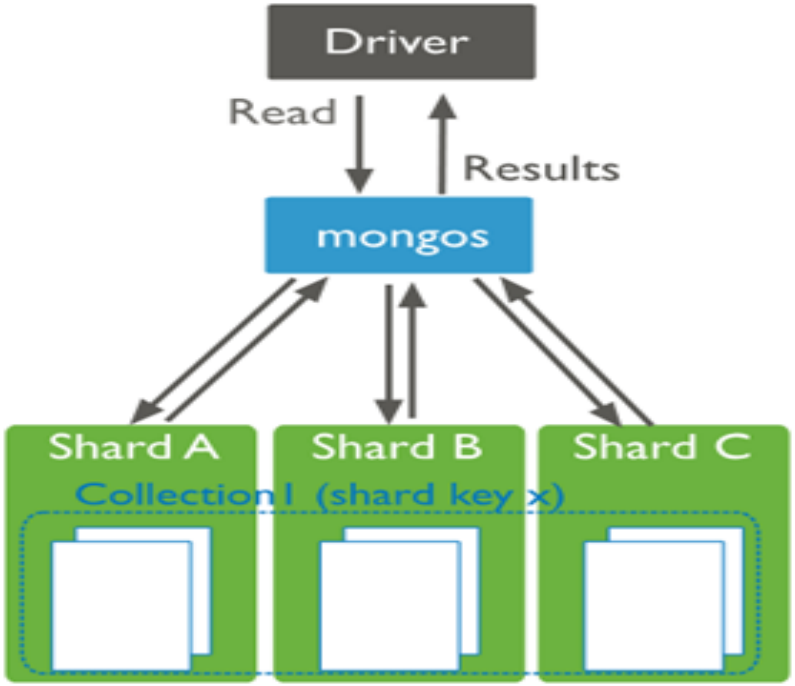
# SHARDING

- Splitting a large dataset across multiple servers (shards).
- A **Shard** is a single server (or replica set) that stores a portion of the total data.
- **Config Servers**: Store metadata about cluster and keeps track which data stored on which shard.
- **Mongos**: Acts as query router. Route requests to correct Shard.



# SHARDING: STEPS

1. Data is divided using a **Shard Key** → A Shard Key is a field used to distribute data. Example: Student_ID, User_ID etc.
2. MongoDB uses shard key to decide Which shard should store the document.
3. Based on shard key value, data is stored in different shards
4. Query router sends request to correct shard
5. Result is returned to client



# SHARDING: TYPES

1. Range-Based Sharding → Data divided based on ranges
2. Hash-Based Sharding → Hash of Shard Key
3. Zone-Based Sharding → Data stored in specific shards based on location.

    Shard A (Zone: "Asia") → Student Records from Asia.

    Shard B (Zone: "Europe") → Student Records from Europe.

## ATOMIC OPERATION

1. An atomic operation is one that is executed completely or not executed at all.
2. In MongoDB, a write operation on a single document is atomic.
3. All write operations (insert, update, delete) are atomic at the document level.
4. If you update multiple fields in one document → it succeeds completely or fails completely.

- db.students.updateOne( { _id: 1 }, { $set: { marks: 90 } } )
- Updating one element in an array is atomic for the entire document.
- MongoDB provides operators that perform atomic updates safely.
- Common Atomic Operators:
- $set → Set field value
- $inc → Increment/Decrement value
- $push → Add to array
- $pull → Remove from array

# ATOMIC OPERATION

1. For operations spanning multiple documents or collections, MongoDB supports transactions.
2. ACID transactions Supported by MongoDB 4.0 and above versions.

```
const session = db.getMongo().startSession();
session.startTransaction();
try {
session.getDatabase("University_DB").Students.updateOne( { roll_no: 101 },
{ $inc: { marks: 5 } } );


session.getDatabase("University_DB").Students.updateOne( { roll_no: 101 },
{ $set: { grade: "A" } } );

session.commitTransaction();
}
catch (error)
{
session.abortTransaction();
}
```

# ATOMIC OPERATION: Multiple Collections

Students Collection: { "roll_no": 101, "name": "Bob", "marks": 35 }

Results Collection: { "roll_no": 101, "status": "Fail" }

```
const session = db.getMongo().startSession();
session.startTransaction();
try
{
  session.getDatabase("University_DB").Students.updateOne(
    { roll_no: 101 },
    { $inc: { marks: 10 } }
  );
```

# ATOMIC OPERATION

```
session.getDatabase("University_DB").Results.insertOne({
    roll_no: 101,
    status: "Pass"
  });
  session.commitTransaction();
}
catch (error)
{
session.abortTransaction();
}
```

# ATOMIC OPERATIONS: IMPORTANCE

- Prevents inconsistent data
- Avoids race conditions
- Ensures safe concurrent updates

# Importing Dataset into Mongodb

1. Download dataset from http://media.mongodb.org/zips.json
2. Install Mongodb Command Tool: https://fastdl.mongodb.org/tools/db/mongodb-database-tools-windows-x86_64-100.14.1.zip
3. Extract above downloaded Zip File
4. After extracting zip file go to bin folder→Copy path as *C:\Users\Downloads\mongodb-database-tools-windows-x86_64-100.14.1\mongodb-database-tools-windows-x86_64-100.14.1\bin*
5. Open cmd prompt then use command
   - cd C:\Users\Siraj\Downloads\mongodb-database-tools-windows-x86_64-100.14.1\mongodb-database-tools-windows-x86_64-100.14.1\bin

# Importing Dataset into Mongodb

6. Then run the following command in cmd prompt
   - *mongoimport --db Countries --collection Zipcodes --file Zips.json*
7. Now open mongosh and use the command to verify imported dataset.

   *show dbs()*

   *show collections()*